

Importador de Pedidos

Tiao Cards · Estudo completo do projeto

Este documento explica, bloco por bloco, cada arquivo do projeto **AppPedidos Phy**. O objetivo é ajudá-lo a entender como o código funciona e aprender os conceitos de programação Python usados na prática.

Arquivos cobertos:

- **LEIA-ME.md** — Documentação de instalação e uso
- **extrator_ocr.py** — Motor de leitura OCR e extração de dados
- **aplicacao_vendas.py** — Interface gráfica e integração com banco Firebird

Sumário

1. Visão Geral do Projeto
2. Conceitos Fundamentais Usados
3. Arquivo: LEIA-ME.md (documentação)
4. Arquivo: extrator_ocr.py
 - 4.1 Importações
 - 4.2 Função extrair_texto_ocr
 - 4.3 Função parsear_dados — bloco Cliente
 - 4.4 Função parsear_dados — bloco Itens
 - 4.5 Função parsear_dados — bloco Totais
 - 4.6 Bloco principal __main__
5. Arquivo: aplicacao_vendas.py
 - 5.1 Importações e configuração do banco
 - 5.2 Classe VendasApp — __init__
 - 5.3 Método conectar_banco
 - 5.4 Método extrair_dados_pdf
 - 5.5 Método processar_pedido — Cliente
 - 5.6 Método processar_pedido — Inserção da Venda
 - 5.7 Método processar_pedido — Inserção dos Itens
6. Fluxo Completo (passo a passo)
7. Pontos de Atenção e Melhorias Sugeridas

1. Visão Geral do Projeto

O **Importador de Pedidos** é uma aplicação desktop feita em Python que resolve um problema prático: quando um pedido chega pelo site *tiaocards.com.br* em formato PDF, alguém precisaria digitar manualmente cada item no sistema de gestão (Firebird/CLIPP). Essa aplicação automatiza esse processo inteiro.

O que ela faz, em linguagem simples:

1. O usuário clica em um botão e escolhe o PDF do pedido.
2. O programa 'lê' o PDF usando OCR (reconhecimento de texto em imagens).
3. O código identifica o cliente, os produtos e os preços no texto extraído.
4. Conecta ao banco de dados Firebird e insere tudo automaticamente.
5. Exibe uma mensagem de sucesso com o número do pedido gerado.

Tecnologias utilizadas:

Biblioteca / Ferramenta	Para que serve
tkinter	Interface gráfica (janelas, botões, labels)
pdf2image + Poppler	Converte cada página do PDF em uma imagem
pytesseract + Tesseract	Lê o texto dentro das imagens (OCR)
re (regex)	Encontra padrões de texto (CPF, referências, preços)
fdb	Conecta e executa comandos no banco de dados Firebird
json	Formata dados estruturados (usado no modo debug)

2. Conceitos Fundamentais Usados

Classes e Objetos

Em Python, uma **classe** é como um molde. Ela define atributos (dados) e métodos (ações). No projeto, `VendasApp` é uma classe que agrupa toda a lógica da janela e das ações do botão.

```
class VendasApp:
    def __init__(self, root): # método construtor
        self.root = root      # atributo: guarda a janela
        self.label = tk.Label(...) # atributo: widget de texto
```

Expressões Regulares (regex)

O módulo `re` permite encontrar padrões dentro de textos. É muito usado aqui para extrair CPF/CNPJ, referências de produto e preços do texto confuso que o OCR produz.

```
# Encontra padrões como 'BLZD-EN090', 'CARD-PT012'...
match = re.search(r'([A-Z]{4}-[A-Z]{2}\d{3})', linha)
#             ^^^^ 4 letras maiúsculas
#             ^^^ traço literal
#             ^^ 2 letras maiúsculas
#             ^^^^ 3 dígitos
```

Contexto 'with' e Conexão ao Banco

O banco Firebird é acessado via `fdb.connect()`. Após executar os comandos, é obrigatório chamar `commit()` para salvar ou `rollback()` para cancelar em caso de erro. O bloco `try/except/finally` garante que a conexão sempre seja fechada.

```
try:
    cur.execute("INSERT INTO TB_...")
    con.commit() # salva no banco
except Exception as e:
    con.rollback() # cancela tudo se der erro
finally:
    con.close() # fecha sempre
```

OCR — Reconhecimento Óptico de Caracteres

O PDF de pedido não é texto puro — é uma imagem. Por isso precisamos de dois passos: (1) **pdf2image** converte cada página em imagem PNG, e (2) **pytesseract** passa essa imagem pelo Tesseract, que reconhece os caracteres e devolve uma string de texto.

3. Arquivo: LEIA-ME.md

O arquivo LEIA-ME.md (README) documenta como instalar e usar o projeto. É o primeiro arquivo que qualquer pessoa deve ler antes de rodar a aplicação.

Conteúdo do arquivo

```
# Importador de Pedidos - Tiao Cards
```

```
Esta aplicação automatiza a entrada de pedidos do site  
`tiaocards.com.br` no seu sistema Firebird.
```

```
## Pré-requisitos
```

1. Python 3.x
2. Tesseract OCR (com dados de idioma "Portuguese")
3. `pip install fdb pyesseract pdf2image`
4. Poppler (necessário para converter PDF em imagem)

```
## Configuração
```

```
Altere a linha 11 de aplicacao_vendas.py:
```

```
'database': 'C:/Caminho/Para/Seu/CLIPP.FDB'
```

```
## Como usar
```

1. Dê um duplo clique em aplicacao_vendas.py
2. Clique em "Selecionar PDF e Processar"
3. Escolha o PDF do pedido
4. Aguarde a mensagem de sucesso!

```
## Lógica de Conversão
```

```
Detecta se a carta é em português (bandeira do Brasil)  
e troca -EN por -PT no código de referência.
```

Explicação dos pré-requisitos

Python 3.x: A linguagem de programação usada. Sem ela, nenhum script .py funciona.

Tesseract OCR: Programa externo (não é biblioteca Python) que realiza o reconhecimento de texto nas imagens. Precisa estar instalado no sistema operacional.

fdb: Driver Python para Firebird. Permite executar SQL diretamente do código.

pdf2image: Biblioteca que usa o Poppler por baixo para converter PDFs em imagens.

Poppler: Ferramenta open-source de renderização de PDF. Precisa estar no PATH do Windows.

4. Arquivo: extrator_ocr.py

Este arquivo contém as funções de 'extração pura': ler o PDF e transformar o texto num dicionário Python organizado. Ele pode ser rodado sozinho para testar a leitura sem abrir a interface gráfica.

4.1 Importações

```
from pdf2image import convert_from_path
import pytesseract
import json
import re
```

from pdf2image import convert_from_path: importa apenas a função necessária do pacote, em vez de importar o pacote inteiro.

pytesseract: wrapper Python do Tesseract. Recebe uma imagem e devolve o texto reconhecido.

json: usado apenas na função `__main__` para exibir o resultado formatado no terminal (debug).

re: expressões regulares — o coração da extração de dados.

4.2 Função `extrair_texto_ocr`

```
def extrair_texto_ocr(caminho_pdf):
    paginas = convert_from_path(caminho_pdf)
    texto_completo = ""
    for i, pagina in enumerate(paginas):
        texto = pytesseract.image_to_string(pagina, lang='por')
        texto_completo += texto + "\n"
    return texto_completo
```

convert_from_path(caminho_pdf): lê o arquivo PDF e retorna uma lista de objetos de imagem PIL, um por página. Se o PDF tiver 3 páginas, a lista terá 3 itens.

enumerate(paginas): percorre a lista dando também o índice. O `i` não é usado aqui, mas é boa prática quando você pode precisar do número da página futuramente.

pytesseract.image_to_string(pagina, lang='por'): passa a imagem para o Tesseract. O parâmetro `lang='por'` usa o modelo de idioma português, melhorando a precisão em palavras com acentos.

texto_completo += texto + "\n": concatena o texto de cada página com uma quebra de linha entre elas.

A função retorna uma única string gigante com todo o texto do PDF.

4.3 Função `parsear_dados` — bloco Cliente

```
def parsear_dados(texto):
    dados = {
        'cliente': {},
        'itens': [],
        'resumo': {}
    }

    linhas = [l.strip() for l in texto.split('\n') if l.strip()]
```

[illegible]

dados = {'cliente': {}, 'itens': [], 'resumo': {}}: cria o dicionário que será preenchido. Note os tipos: dict para cliente e resumo, list para itens (pode haver vários produtos).

if "**Destinatário**" in linha: o OCR geralmente lê a linha do destinatário como uma linha longa contendo essa palavra-chave.

.split(",")[0]: o nome do cliente pode vir como 'João Silva, CPF: 123...' — pegar o índice 0 pega só o nome.

```

match_ref = re.search(r"([A-Z]{4}-[A-Z]{2}\d{3})", linha)
if match_ref:
    ref_original = match_ref.group(1)

# Quantidade: procura 'Nx' nas últimas 3 linhas
qtd = 1
for k in range(i, max(-1, i-3), -1):
    match_qtd = re.search(r"(\d+)x", linhas[k])
    if match_qtd:
        qtd = int(match_qtd.group(1))
        break

# Conversão EN → PT se contexto indicar idioma PT
ref_final = ref_original
contexto = linha + (linhas[i-1] if i > 0 else "")
if "[PT]" in contexto or "PT]" in contexto:
    if "-EN" in ref_original:
        ref_final = ref_original.replace("-EN", "-PT")

# Preço unitário nas próximas 4 linhas
preco_unit = 0.0
for j in range(i, min(i+4, len(linhas))):
    precos = re.findall(r"R\$\$s*([\d.]+)", linhas[j])
    if len(precos) >= 2:
        preco_unit = float(precos[-2].replace('.', ',').replace(',', '.'))
        break

if not any(item['referencia_original'] == ref_original
           for item in dados['itens']):
    dados['itens'].append({
        'quantidade': qtd,
        'referencia_original': ref_original,

```

```

        'referencia_final': ref_final,
        'preco_unitario': preco_unit,
        ...
    })

```

r"([A-Z]{4}-[A-Z]{2})\d{3}": regex para capturar referências do tipo BLZD-EN090 — exatamente 4 letras, traço, 2 letras, 3 dígitos.

range(i, max(-1, i-3), -1): loop de trás para frente, partindo da linha atual (i) até 3 linhas acima. Isso porque a quantidade (ex: '2x') pode estar numa linha antes do código do produto.

Lógica EN→PT: alguns produtos têm versão em inglês e em português com referências diferentes (ex: BLZD-EN090 vs BLZD-PT090). Se o OCR leu '[PT]' perto do produto, troca automaticamente o sufixo.

re.findall: diferente de re.search, retorna TODOS os matches. precos[-2] pega o penúltimo valor encontrado (preço unitário), pois o último costuma ser o total da linha.

not any(...): evita duplicatas. O OCR às vezes lê o mesmo produto duas vezes (ex: em linhas adjacentes). O any() verifica se a referência já está na lista antes de adicionar.

4.5 Função parsear_dados — bloco Totais

```

if "Valor dos Itens:" in linha:
    v = re.search(r"R\$\s*([\d,.]+)", linha)
    if v: dados['resumo']['valor_itens'] = float(
        v.group(1).replace('.', '').replace(',', '.'))
if "Frete:" in linha:
    v = re.search(r"R\$\s*([\d,.]+)", linha)
    if v: dados['resumo']['valor_frete'] = float(...)
if "Valor Total:" in linha:
    v = re.search(r"R\$\s*([\d,.]+)", linha)
    if v: dados['resumo']['valor_total'] = float(...)

```

Cada if procura uma linha específica no texto do OCR.

.replace('.', '').replace(',', '.): converte o formato brasileiro (R\$ 1.234,56) para o formato que o Python entende como float (1234.56). Primeiro remove os pontos de milhar, depois troca a vírgula decimal por ponto.

4.6 Bloco principal __main__

```

if __name__ == "__main__":
    caminho = "/home/ubuntu/projeto_firebird/pdf pedido.pdf"
    texto = extrair_texto_ocr(caminho)
    resultado = parsear_dados(texto)
    print(json.dumps(resultado, indent=2, ensure_ascii=False))

```

if __name__ == "__main__": este bloco só roda quando você executa o arquivo diretamente (python extrator_ocr.py). Se outro arquivo importar este módulo, o bloco é ignorado. É o ponto de entrada para testes.

json.dumps(..., indent=2, ensure_ascii=False): formata o dicionário como JSON bonito com indentação de 2 espaços. ensure_ascii=False garante que acentos sejam impressos corretamente em vez de códigos \uXXXX.

5. Arquivo: aplicacao_vendas.py

Este é o arquivo principal. Ele combina a extração OCR do arquivo anterior com a interface gráfica (tkinter) e a integração com o banco Firebird.

5.1 Importações e Configuração do Banco

```
import os
import re
import json
import fdb
import pytesseract
from pdf2image import convert_from_path
import tkinter as tk
from tkinter import filedialog, messagebox

DB_CONFIG = {
    'database': 'C:\\Users\\junin\\...\\CLIPP.FDB',
    'user': 'SYSDBA',
    'password': 'masterkey',
    'charset': 'WIN1252'
}
```

fdb: driver Python para o banco de dados Firebird.

tkinter: biblioteca padrão do Python para criar interfaces gráficas. Não precisa instalar — já vem com o Python.

filedialog: módulo do tkinter para abrir a janela 'Abrir Arquivo'.

messagebox: módulo do tkinter para exibir caixas de diálogo (erro, aviso, informação).

DB_CONFIG: dicionário com as configurações de conexão. Usar um dicionário facilita passar tudo de uma vez com ****DB_CONFIG**. O **charset='WIN1252'** é essencial para que acentos do português sejam lidos corretamente do banco.

5.2 Classe VendasApp — `__init__`

```
class VendasApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Importador de Pedidos - Tiao Cards")
        self.root.geometry("500x300")

        self.label = tk.Label(root,
                               text="Selecione o PDF do pedido para importar", pady=20)
        self.label.pack()

        self.btn_select = tk.Button(root,
                                     text="Selecionar PDF e Processar",
                                     command=self.processar_pedido,
                                     height=2, width=30)
        self.btn_select.pack(pady=10)

        self.status = tk.Label(root, text="Aguardando...", fg="blue")
        self.status.pack(pady=20)
```

__init__(self, root): construtor da classe. É chamado uma vez quando criamos `VendasApp(root)`. O parâmetro `root` é a janela principal do tkinter.

self.root.geometry("500x300"): define o tamanho inicial da janela em pixels (largura x altura).

tk.Label: widget de texto estático (não editável pelo usuário).

tk.Button(..., command=self.processar_pedido): o parâmetro `command` define qual função será chamada ao clicar. Note que passa `self.processar_pedido` sem parênteses — estamos passando a referência da função, não chamando ela.

.pack(): posiciona o widget na janela de cima para baixo. É o gerenciador de layout mais simples do tkinter.

self.status: label especial que será atualizado durante o processamento para informar o progresso ao usuário.

5.3 Método conectar_banco

```
def conectar_banco(self):
    try:
        return fdb.connect(**DB_CONFIG)
    except Exception as e:
        messagebox.showerror("Erro de Conexão",
                             f"Não foi possível conectar ao Firebird:\n{e}")
    return None
```

fdb.connect(DB_CONFIG):** o operador `**` desempacota o dicionário como argumentos nomeados. Equivale a escrever: `fdb.connect(database='...', user='SYSDBA', ...)`

return None: se a conexão falhar, retorna `None`. O método `processar_pedido` verifica isso com `if not con:` `return`.

5.4 Método extrair_dados_pdf

```
def extrair_dados_pdf(self, caminho_pdf):
    self.status.config(text="Lendo PDF (OCR)...", fg="orange")
    self.root.update()

    paginas = convert_from_path(caminho_pdf)
    texto_completo = ""
    for pagina in paginas:
        texto_completo += pytesseract.image_to_string(
            pagina, lang='por') + "\n"

    dados = {'cliente': {}, 'itens': [], 'resumo': {}}
    linhas = [l.strip() for l in texto_completo.split('\n') if l.strip()]
    # ... (mesma lógica do extrator_ocr.py)
    return dados
```

self.status.config(text=..., fg="orange"): atualiza o texto e a cor do label de status. `fg` é a cor da fonte (foreground = frente).

self.root.update(): força o tkinter a redesenhar a janela *imediatamente*. Sem isso, a interface 'trava' durante o OCR (que pode demorar alguns segundos) e o usuário não vê a mensagem 'Lendo PDF...!'.

A lógica de extração é idêntica ao extrator_ocr.py — o código foi duplicado aqui para que o arquivo seja autocontido.

5.5 Método processar_pedido — Cliente

```
def processar_pedido(self):
    caminho_pdf = filedialog.askopenfilename(
        filetypes=[("Arquivos PDF", "*.pdf")])
    if not caminho_pdf: return

    dados = self.extrair_dados_pdf(caminho_pdf)
    if not dados['itens']:
        messagebox.showwarning("Aviso",
            "Nenhum item encontrado no PDF.")
        return

    con = self.conectar_banco()
    if not con: return

    try:
        cur = con.cursor()
        doc = dados['cliente'].get('documento')
        id_cliente = None

        # Busca em PF (pessoa física)
        cur.execute(
            "SELECT ID_CLIENTE FROM TB_CLI_PF WHERE CPF = ?", (doc,))
        row = cur.fetchone()
        if row:
            id_cliente = row[0]
        else:
            # Busca em PJ (pessoa jurídica)
            cur.execute(
                "SELECT ID_CLIENTE FROM TB_CLI_PJ WHERE CNPJ = ?", (doc,))
            row = cur.fetchone()
            if row: id_cliente = row[0]

        if not id_cliente:
            cur.execute(
                "INSERT INTO TB_CLIENTE (NOME, ID_PAIS) "
                "VALUES (?, '0105') RETURNING ID_CLIENTE",
                (dados['cliente']['nome'],))
            id_cliente = cur.fetchone()[0]
            if len(doc) <= 11:
                cur.execute(
                    "INSERT INTO TB_CLI_PF (ID_CLIENTE, CPF) VALUES (?, ?)",
                    (id_cliente, doc))
            else:
                cur.execute(
                    "INSERT INTO TB_CLI_PJ (ID_CLIENTE, CNPJ) VALUES (?, ?)",
                    (id_cliente, doc))
```

filedialog.askopenfilename(filetypes=[...]): abre a janela nativa do sistema operacional para escolher um arquivo. O filetypes filtra para mostrar só PDFs.

cur = con.cursor(): cria um cursor, que é o objeto usado para executar comandos SQL.

cur.execute("SELECT ... WHERE CPF = ?", (doc,)): o ? é um placeholder — o fdb substitui pelo valor de (doc,) de forma segura, prevenindo SQL injection. Note a vírgula: (doc,) cria uma tupla com um elemento; (doc) seria só parênteses.

RETURNING ID_CLIENTE: cláusula Firebird que retorna o valor gerado pelo banco logo após o INSERT, sem precisar fazer um SELECT adicional.

len(doc) <= 11: CPF tem 11 dígitos, CNPJ tem 14. Isso determina em qual tabela salvar o documento do cliente.

5.6 Método processar_pedido — Inserção da Venda

```
cur.execute("""
    INSERT INTO TB_NFVENDA_2
        (ID_CLIENTE, DT_EMISSAO, TOTALNOTA, TOTALPRODUTOS,
         ID_NATOP, ID_FMAPGTO, ID_PARCELA, STATUS,
         ENT_SAI, NF_NUMERO, NF_SERIE, NF_MODELO)
    VALUES
        (?, CURRENT_DATE, ?, ?, 1, 1, 1, 'A', 'S', 0, '1', '01')
    RETURNING ID_NFVENDA
""", (id_cliente,
     dados['resumo'].get('total', 0),
     dados['resumo'].get('total', 0)))
id_venda = cur.fetchone()[0]
```

"""...(SQL)...": string multilinha (triple-quoted). Permite escrever o SQL em múltiplas linhas para melhor leitura.

CURRENT_DATE: função do Firebird que insere a data de hoje automaticamente — sem precisar passar no Python.

STATUS = 'A': provavelmente 'Aberto' ou 'Ativo', conforme convenção do sistema CLIPP.

ENT_SAI = 'S': indica Saída (venda), diferente de 'E' (Entrada).

.get('total', 0): acessa o dicionário com valor padrão 0 se a chave não existir — mais seguro que `dados['resumo']['total']`, que lançaria `KeyError` se o OCR não encontrou o total.

5.7 Método processar_pedido — Inserção dos Itens

```
for item in dados['itens']:
    cur.execute(
        "SELECT ID_IDENTIFICADOR FROM TB_EST_PRODUTO_2 "
        "WHERE REFERENCIA = ?",
        (item['referencia'],))
    prod_row = cur.fetchone()

    if prod_row:
        id_prod = prod_row[0]
        cur.execute("""
            INSERT INTO TB_NFV_ITEM_2
                (ID_NFVENDA, ID_IDENTIFICADOR, QTD_ITEM,
                 VLR_UNIT, VLR_TOTAL, CFOP, NUM_ITEM,
                 VLR_FRETE, INCLUIR_FATURA)
            VALUES (?, ?, ?, ?, ?, '5102', 1, 0, 'S')
""", (id_venda, id_prod,
```

```

        item['quantidade'],
        item['preco_unitario'],
        item['quantidade'] * item['preco_unitario']))
    else:
        print(f"Produto não encontrado: {item['referencia']}")

    con.commit()
    self.status.config(text="Pedido importado com sucesso!", fg="green")
    messagebox.showinfo("Sucesso",
        f"Pedido importado com ID: {id_venda}")

except Exception as e:
    con.rollback()
    messagebox.showerror("Erro", f"Erro no banco:\n{e}")
finally:
    con.close()

```

for item in dados[itens]: itera sobre a lista de itens extraídos do PDF. Para cada produto, primeiro busca o ID no cadastro.

SELECT ID_IDENTIFICADOR: busca o produto pela referência lida no PDF. Se não encontrar, registra no terminal mas não interrompe o processo.

CFOP = '5102': código fiscal de operação para venda de mercadoria adquirida ou recebida de terceiros — padrão no Brasil para vendas dentro do estado.

item['quantidade'] * item['preco_unitario']: calcula o total do item diretamente no Python antes de inserir.

con.commit(): só após inserir TODOS os itens com sucesso. Isso garante que, se um item der erro, nada é salvo parcialmente — ou entra tudo ou não entra nada (atomicidade).

6. Fluxo Completo (passo a passo)

Veja o que acontece desde o clique do usuário até a confirmação:

Usuário clica em "Selecionar PDF e Processar"



`filedialog.askopenfilename()` → Janela nativa do Windows para escolher o PDF



`extrair_dados_pdf(caminho_pdf)`

■■■■ `convert_from_path(pdf)` → lista de imagens PIL

■■■■ `pytesseract.image_to_string(img, lang='por')` → texto bruto

■■■■ regex sobre texto → dict{cliente, itens, resumo}



`conectar_banco()`

■■■■ `fdb.connect(**DB_CONFIG)` → objeto de conexão



Buscar ou Criar Cliente

■■■■ `SELECT` em `TB_CLI_PF` (CPF)

■■■■ `SELECT` em `TB_CLI_PJ` (CNPJ)

■■■■ `INSERT TB_CLIENTE + PF/PJ` (se não existir)



`INSERT TB_NFVENDA_2` → `id_venda` = novo ID gerado



Para cada item:

■■■■ `SELECT TB_EST_PRODUTO_2` (busca ID pelo código de referência)

■■■■ `INSERT TB_NFV_ITEM_2` (quantidade, preço unitário, total)



`con.commit()` → tudo salvo no banco



`messagebox.showinfo("Sucesso")` → exibe ID do pedido gerado

7. Pontos de Atenção e Melhorias Sugeridas

Pontos que precisam de atenção

1. **Senha hardcoded:** a senha masterkey está diretamente no código. Em produção, use variáveis de ambiente ou um arquivo de configuração não versionado.
2. **Campos fixos no INSERT de venda:** ID_NATOPE=1, ID_FMAPGTO=1, ID_PARCELA=1 são valores hardcoded — certifique-se que esses IDs existem no seu banco.
3. **self.root.update() bloqueia:** o OCR roda na thread principal. Para PDFs grandes, a janela pode não responder durante o processamento. A solução avançada seria usar threading.
4. **referencia_original não existe no dict do aplicacao_vendas.py:** o check de duplicata usa item['referencia_original'] mas o dict criado não tem essa chave — pode gerar um KeyError.

Melhorias sugeridas

- Adicionar uma barra de progresso (ttk.Progressbar) durante o OCR.
- Exibir os itens encontrados numa tabela antes de confirmar o import.
- Mover as credenciais do banco para um arquivo config.ini.
- Implementar log de erros em arquivo para facilitar debugging.
- Usar threading.Thread para não bloquear a interface durante o OCR.

Fim do documento. Bons estudos!